PROJECT PRESENTATION & VIVA DEFENSE GUIDE

DANGEROUS ADVENTURE

How to Explain the Architecture, Physics Engine & Technical Innovation to Your Teacher

Topic: Modern 60 FPS Platformer Game Engine | Tech: Vanilla JS (ES6) + Canvas + Web Audio | **Score: 16/16 Test Suites Passed**

1. The 30-Second Elevator Pitch (Start with This)

"Good morning/afternoon, Sir/Ma'am. My project is Dangerous Adventure, a full-featured 2D platformer game engine built entirely in pure Vanilla JavaScript and HTML5 Canvas with zero external dependencies. It re-engineers the classic DOS Dangerous Dave game with smooth 60 FPS sub-pixel physics, real-time procedural 8-bit sound synthesis, dual combat mechanics, and a multi-level campaign validated by 16 automated test suites."

2. The 5 Core Engineering Pillars (Technical Highlights)

1 — Custom Axis-Separated AABB Physics & Game Loop

Explain: "Instead of third-party physics libraries, I wrote a custom continuous Axis-Aligned Bounding Box (AABB) collision solver. Movement is decoupled from monitor frame rates using fixed-time delta-time (dt) integration. I also engineered Coyote Time (ledge forgiveness) and Jump Buffering for precision platforming."

2 — Procedural Web Audio API Sound Synthesizer

Explain: "The game requires zero external .mp3 audio files. Using the browser's native Web Audio API, I synthesize square, triangle, and noise waves procedurally on-the-fly for blaster shots, coin chimes, explosions, and fanfares with zero download lag."

3 — Dual-Mode Combat & Smart Enemy AI System

Explain: "Classic Dave only had one-hit deaths. I implemented a dual combat system where Dave can fire high-velocity Plasma Bolts (F key) OR perform a Super Mario-style Enemy Stomp from above for a bounce and +200 points. The enemies feature intelligent patrol AI with ledge detection."

4 — Modular ES6 Architecture & Finite State Machine (FSM)

Explain: "The game is architected around a strict Finite State Machine governing 9 distinct states (Main Menu, Level Select, Instructions, Settings, Playing, Paused, Game Over, Level Complete, and Victory) with decoupled single-responsibility modules."

5 — 16 Automated Headless Test Suites (Quality Assurance)

Explain: "To guarantee rock-solid stability and zero regressions, I built a test harness with 16 automated test suites executed via Node.js verifying physics, hitboxes, door logic, score persistence, and HUD rendering with 100% pass rate."

3. Live Project Demonstration Script (What to Show & Say)

Step 1: Main Menu & Settings
Show the hologram pedestal and 60 FPS menu. Navigate into Settings to show the dynamic [ON] / [OFF] sound/CRT toggles and explain that all settings persist across games.

Step 2: Level 1 & Physics Demo
Click PLAY or LEVEL 1. Show the smooth variable jump (light tap vs. high jump), coin collection, and smooth camera tracking. Show that Dave never snags on wall seams.

Step 3: Combat (Blaster & Stomp)
Press F to shoot a robot with the Plasma Blaster. Then jump on top of a patrolling guard to demonstrate the Stomp kill bounce (+200 pts).

Step 4: Checkpoints & Trophy Goal
Walk into a green Checkpoint Beacon to show auto-saving. Grab the Golden Trophy (HUD changes to [ACQUIRED]) and enter the Exit Door to complete the level.

Step 5: Automated Test Execution
Open the terminal and run "node scratch/run_all_tests.js" to show the examiner 16/16 test suites passing in under 2 seconds.

4. Likely Teacher / Viva Questions & Exact Answers

Q1: Why did you build this in Vanilla JS instead of using Phaser, Unity, or React?

Answer: "Building in Vanilla JS demonstrates a deep understanding of core computer science fundamentalsâ game loops, vector math, collision physics, and audio synthesisâ without relying on heavy black-box engine abstractions. It also results in near-instant load times (<50ms) and minimal memory footprint."

Q2: How does your collision detection work without tile clipping?

Answer: "I use Axis-Separated Continuous AABB (Axis-Aligned Bounding Box) testing. In each frame, horizontal displacement (dx) is tested and clamped against solid map tiles first, followed by vertical displacement (dy). This guarantees Dave never snags on corners or falls through floors."

Q3: How does your audio work without importing MP3/WAV files?

Answer: "I utilize the Web Audio API AudioContext. When an event fires (e.g. blaster shot), the engine instantiates an OscillatorNode, sets its frequency curve (880Hz to 110Hz), routes it through a GainNode envelope, and plays the procedural chiptune wave with zero network latency."

Q4: What is delta-time (dt) and why is it important in your game loop?

Answer: "Delta-time is the exact elapsed time between consecutive animation frames. Multiplying velocity by dt ensures that Dave and enemies move at the exact same physical speed regardless of whether the user is playing on a 60Hz, 120Hz, or 144Hz display."

Q5: How did you solve the text readability and layout bugs in retro canvas?

Answer: "Retro canvas bitmap fonts blur when scaled down below 6px. I engineered a Dual-Layer Typography System: retro fonts (Press Start 2P) are preserved for arcade titles, while body text and instructions use clean, high-contrast typography (Outfit 8.5px bold) with bounded coordinate clipping."

Q6: How do you verify that changes don't break existing levels or physics?

Answer: "I created 16 automated headless test suites that simulate user inputs, physics ticks, and mock canvas draw commands. Running ‘node scratch/run_all_tests.js‘ executes and validates all 16 test suites in seconds."